

팍리스 실무형 일경험 프로그램

SDV 아키텍처 기반 자율주행 통합 시스템 구현 실습



9일차

02 Sensing Zone과 HPC Zone 연동 구현

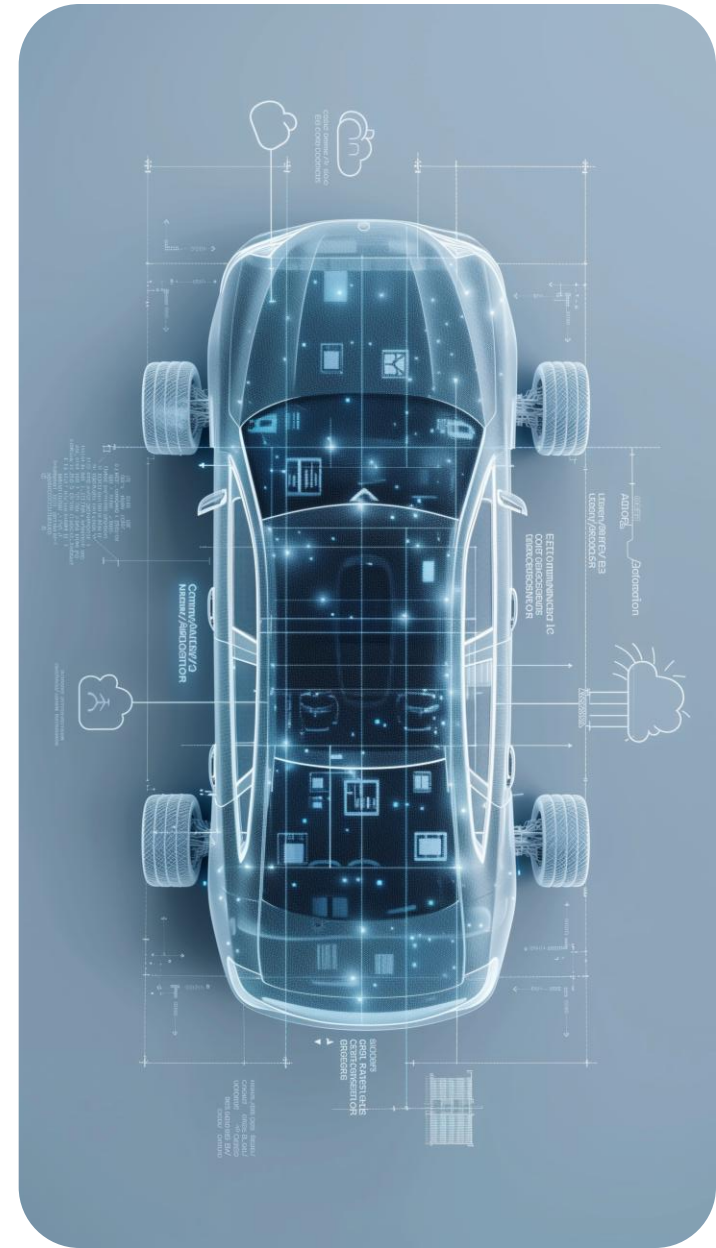
Telechips TOPST



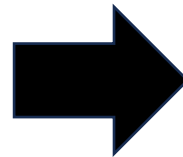
목차

Table of Contents

Panel App 동작 구조	01
Panel App 구현	02
Panel App 구현 확인	03

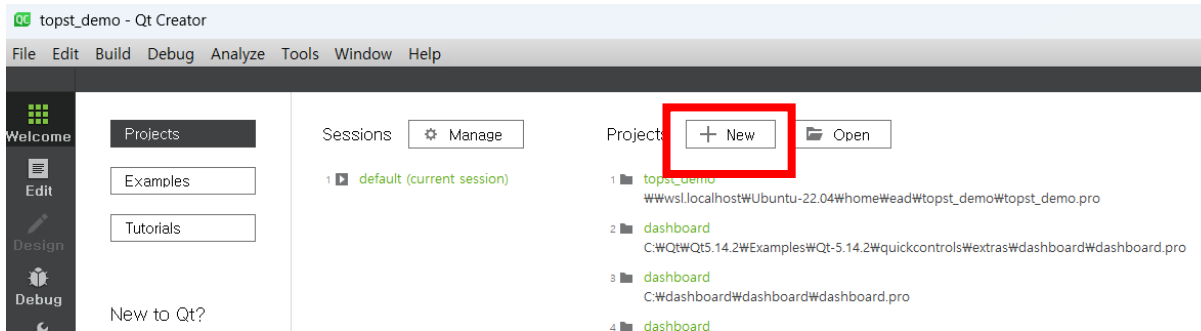


Panel App 동작 구조

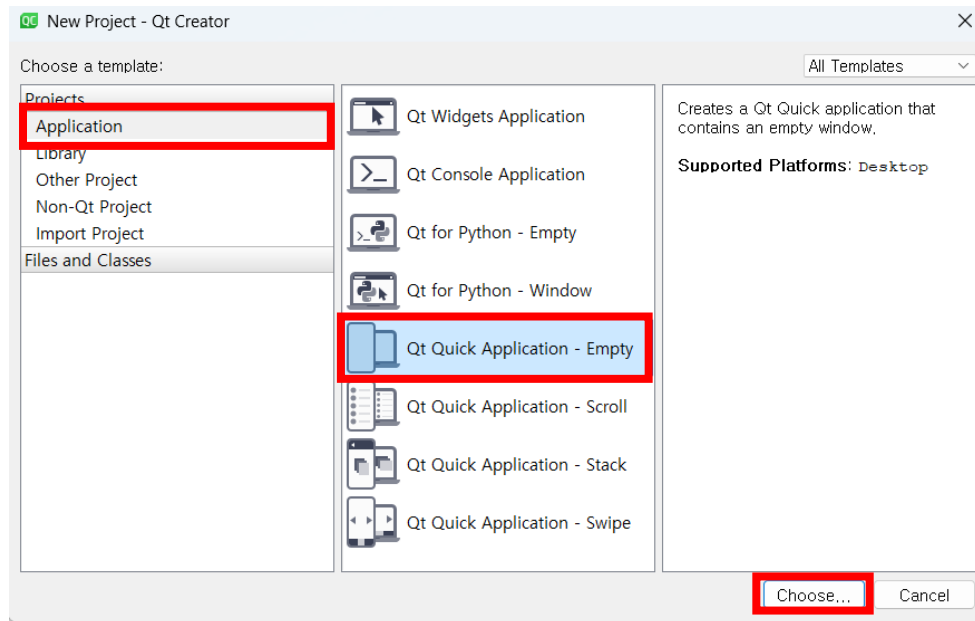


- Yolov8 모델 추론
- 추론된 바운딩 박스의 좌표 값으로 중심 좌표 계산
- 중심 좌표 위치에 따라 특정 위치 매핑 -> 차량 모델 출력

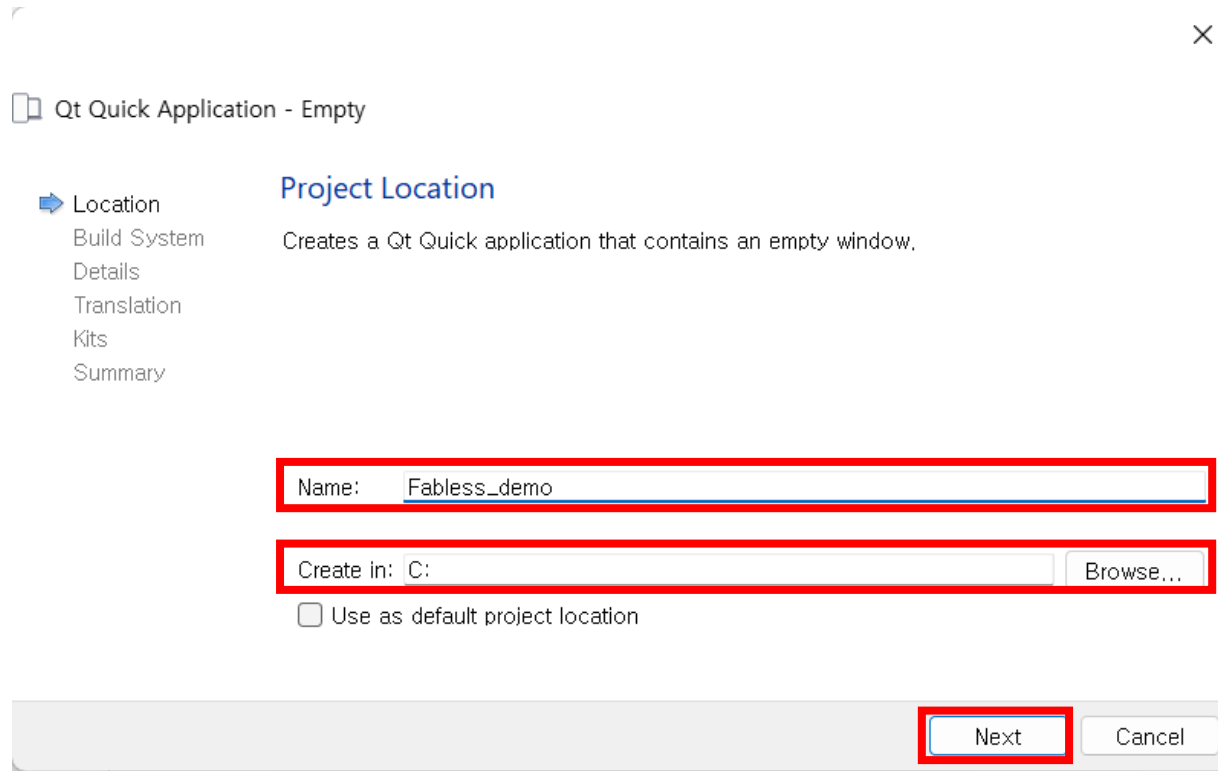
Panel App 구현



- QT creator 실행
- New 탭 선택
- Application -> Qt Quick Application 선택
- Choose 클릭



Panel App 구현



Qt Quick Application - Empty

Location
Build System
Details
Translation
Kits
Summary

Project Location
Creates a Qt Quick application that contains an empty window.

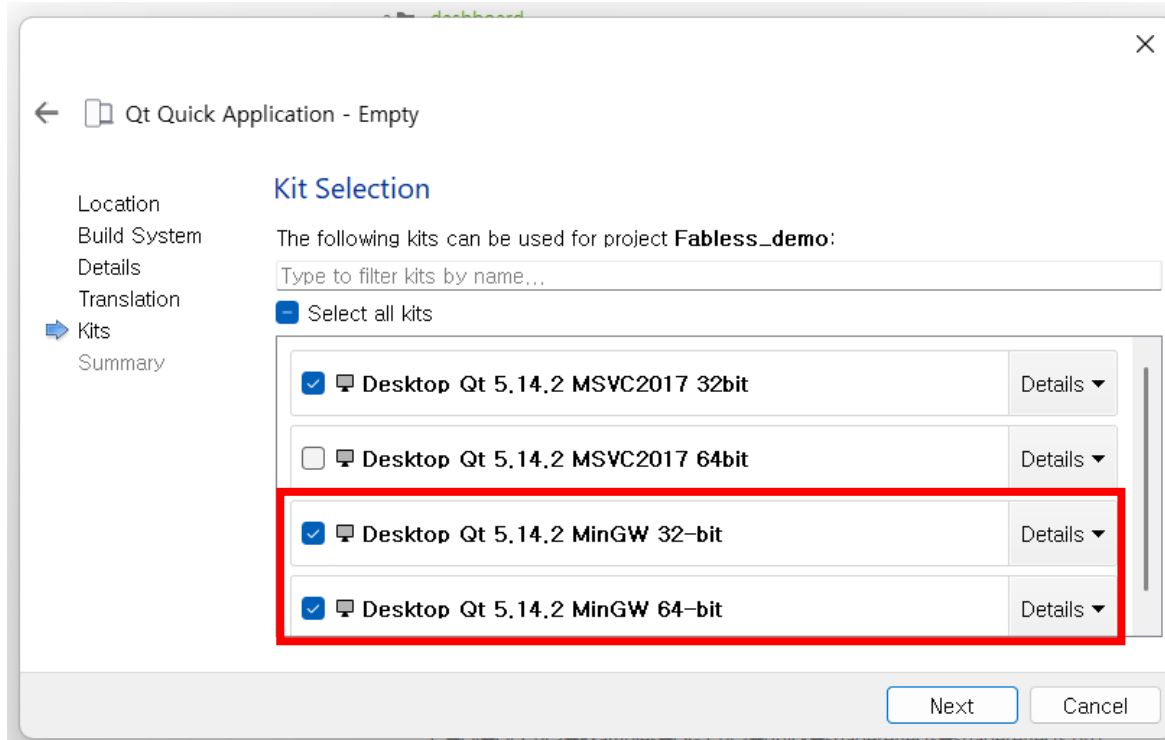
Name:

Create in:

☐ Use as default project location

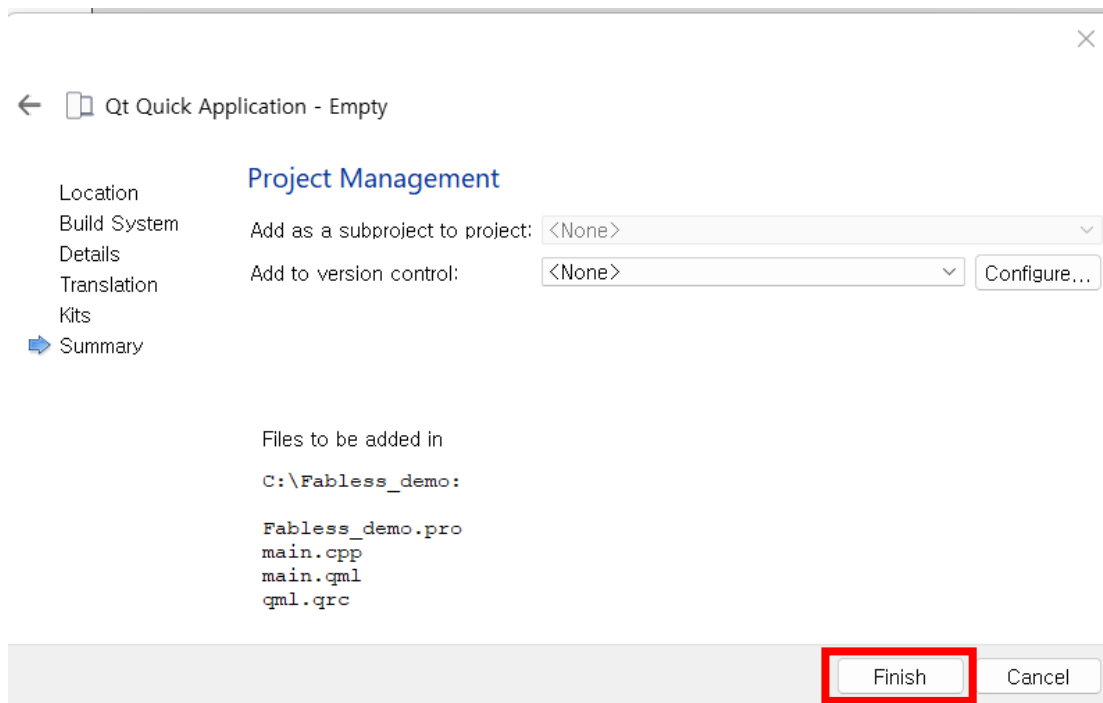
- Name : Fabless_demo 수정
- Create in : 로컬 디스크 C
- Next -> Next -> Next
- ->Next

Panel App 구현



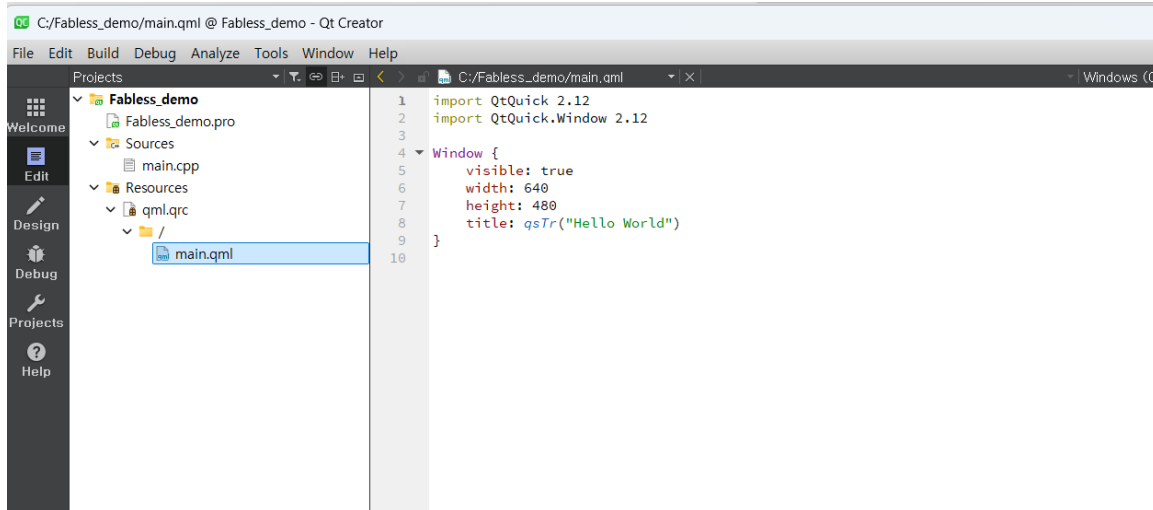
- Desktop QT 5.14.2 MinGW 32-bit / 64-bit 활성화
- Next 클릭

Panel App 구현



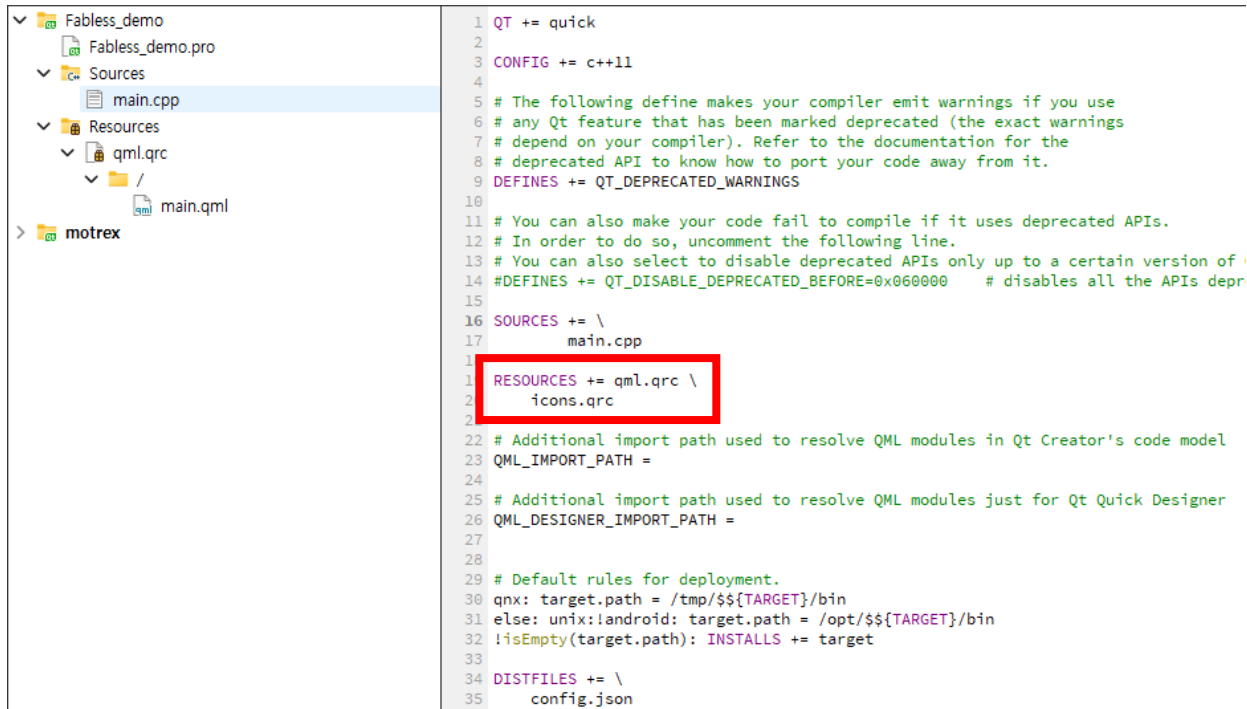
- Finish 클릭

Panel App 구현



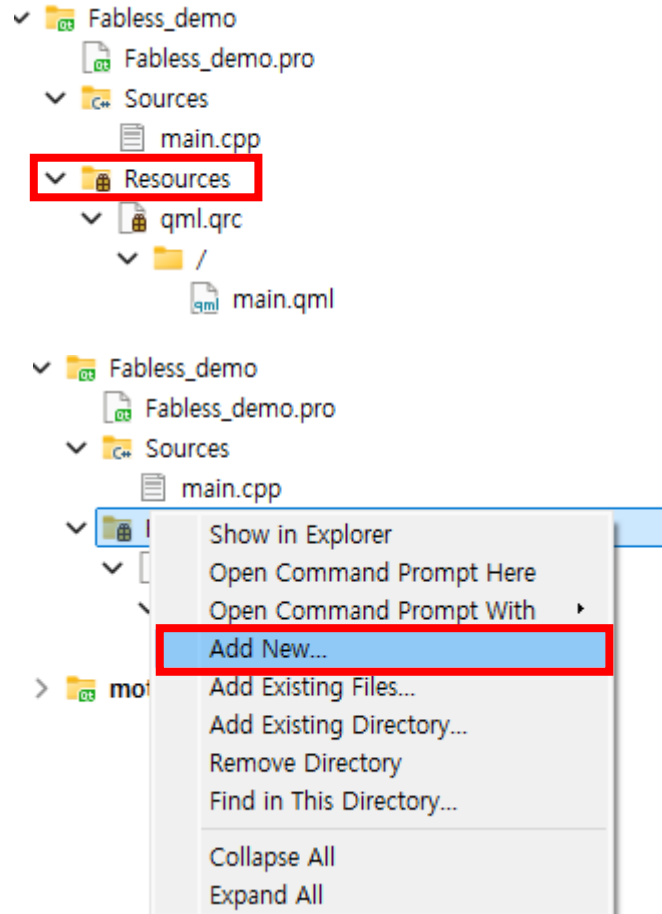
- 해당 화면 확인

Panel App 구현



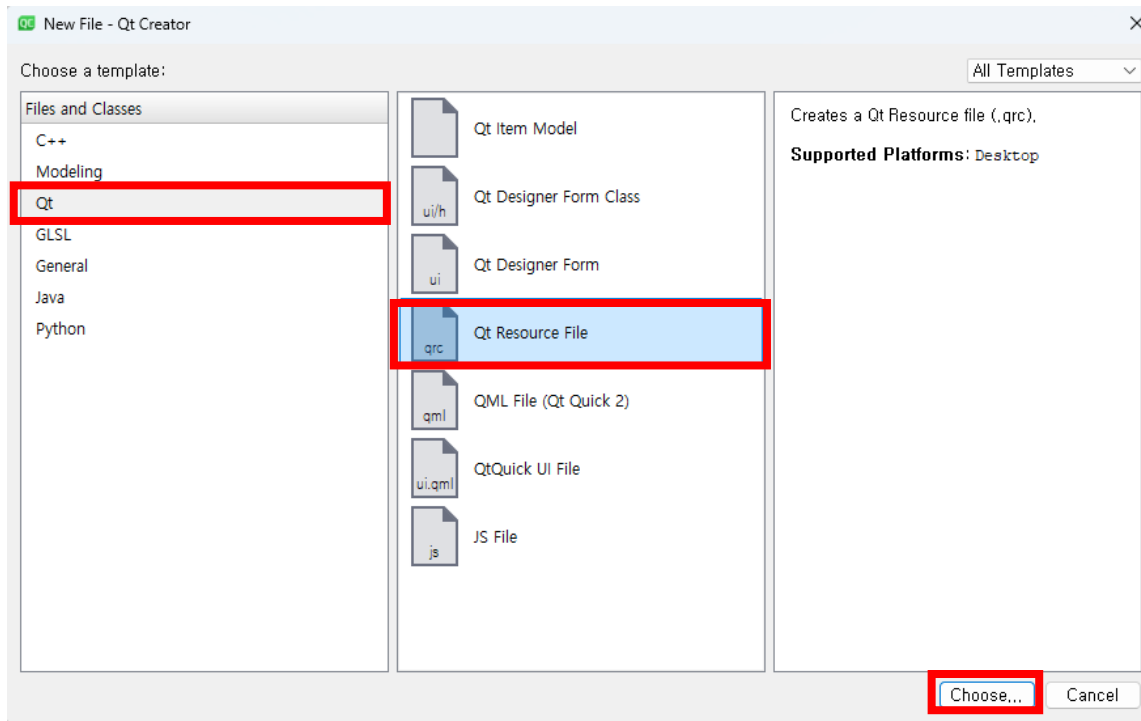
- Fabless_demo.pro 선택
- RESOURCES에 icons.qrc 추가

Panel App 구현



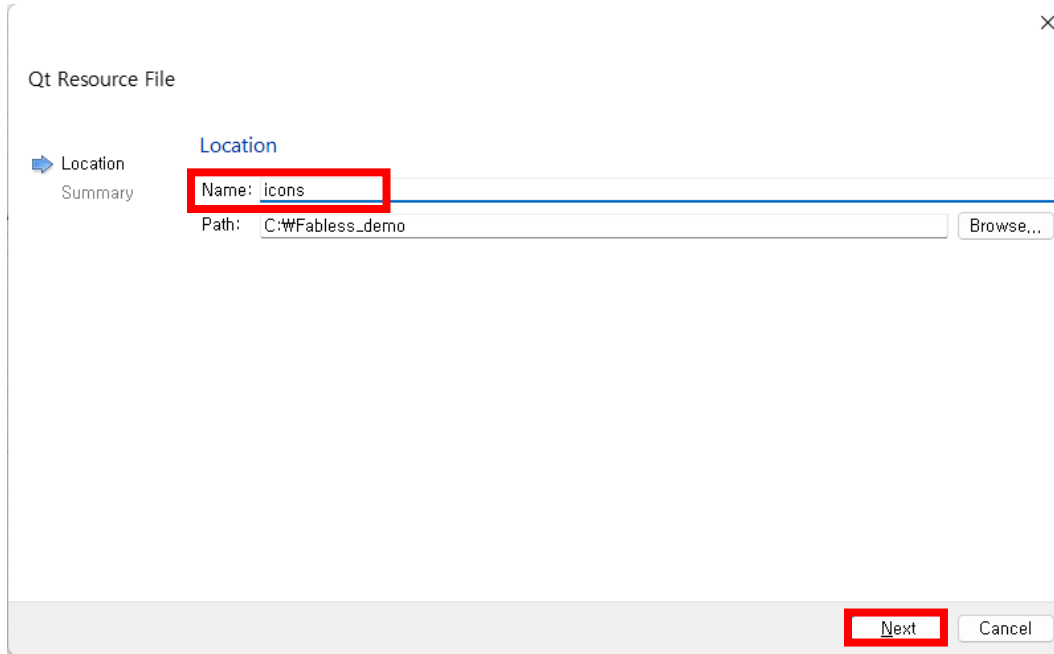
- Resources 우클릭
- Add New... 선택

Panel App 구현



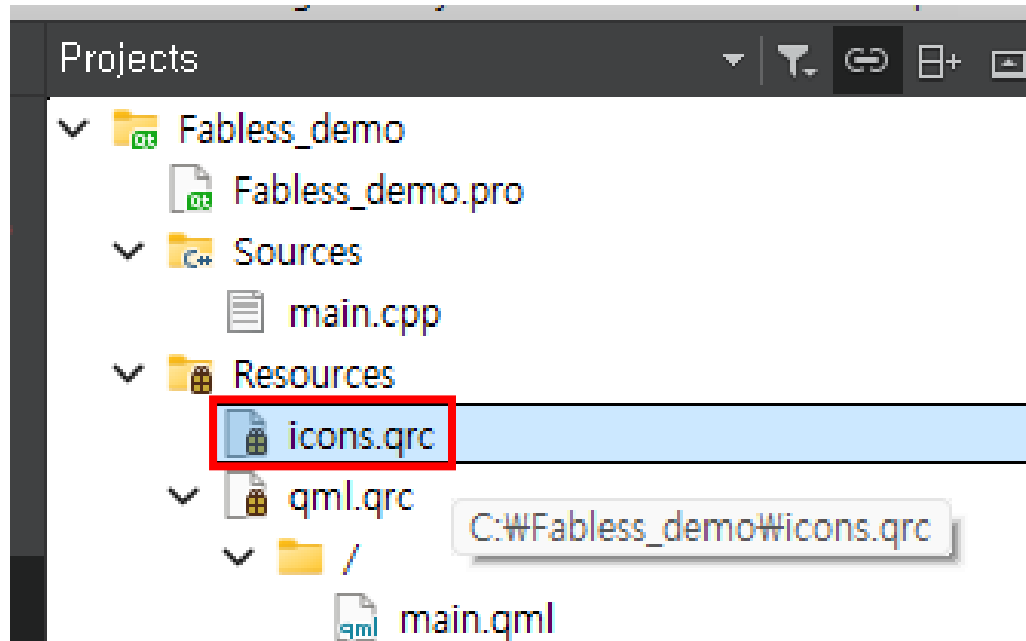
- Qt -> Qt Resources File 선택
- Choose 클릭

Panel App 구현



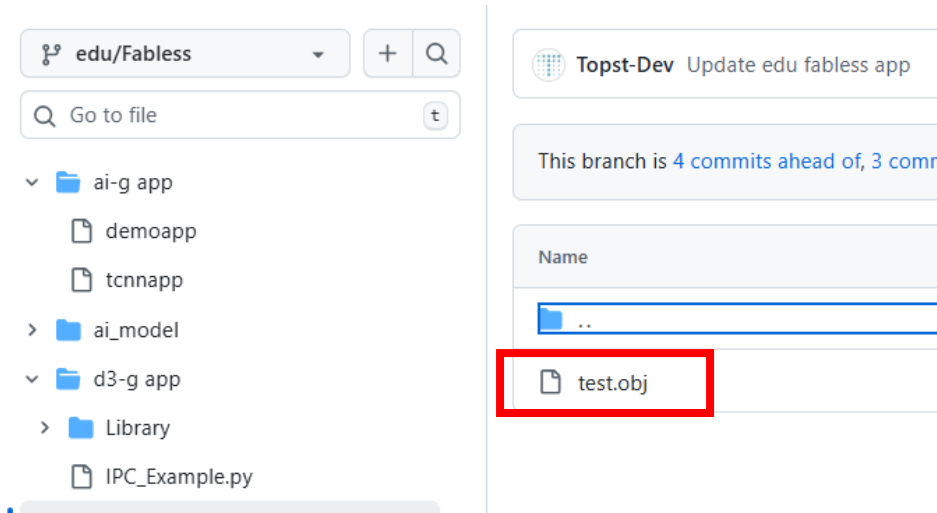
- Name – icons 입력
- Next -> Finish

Panel App 구현

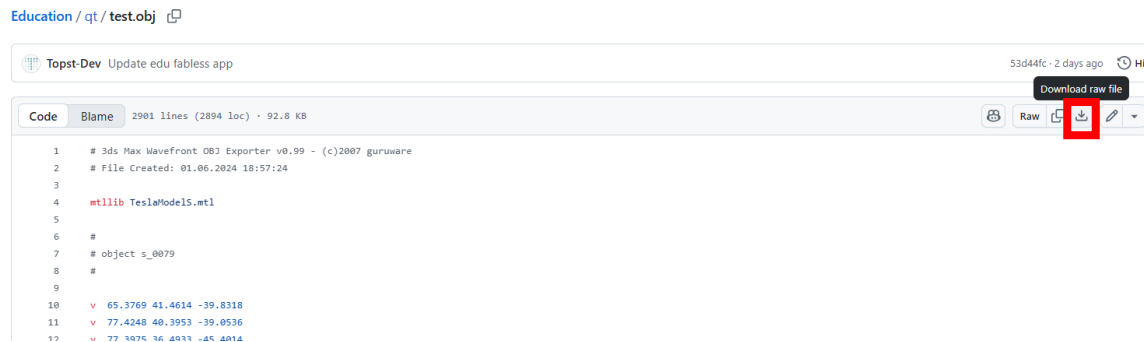


- icons.qrc 확인

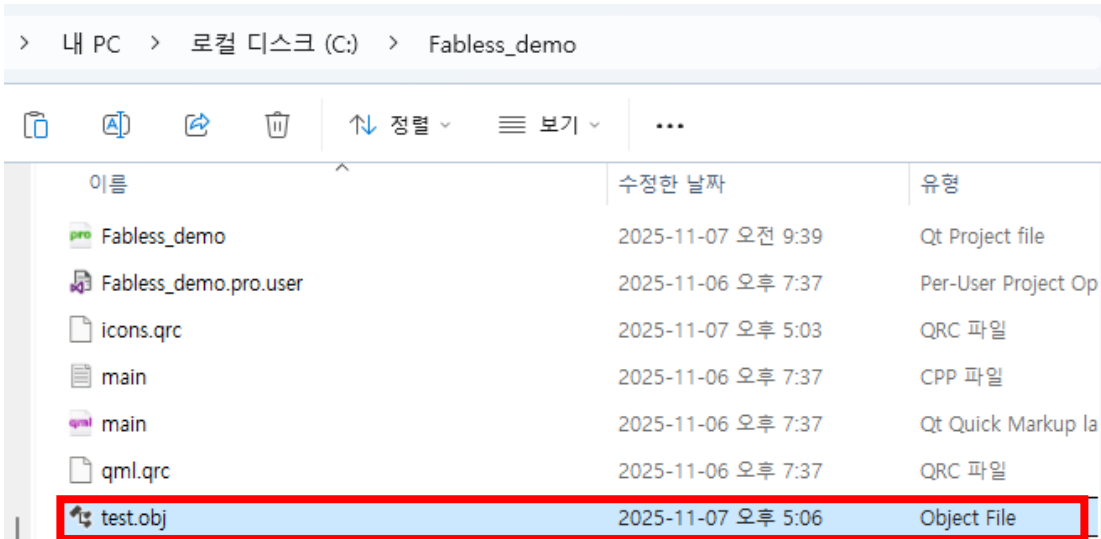
Panel App 구현



- [Education/qt at edu/Fabless · topst-development/Education](#)
- 해당 링크 접속
- test.obj 선택
- 다운로드 버튼 클릭

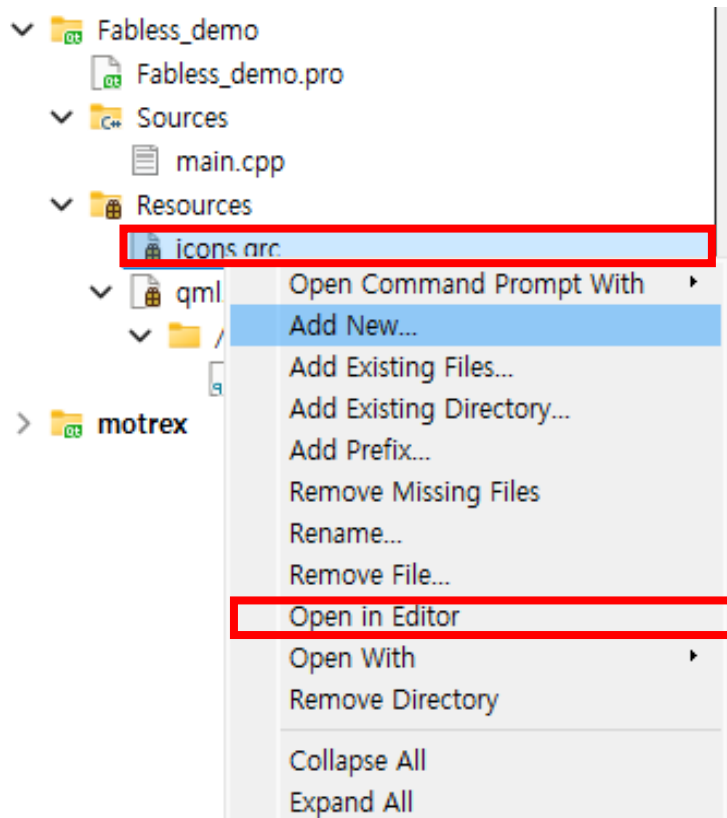


Panel App 구현



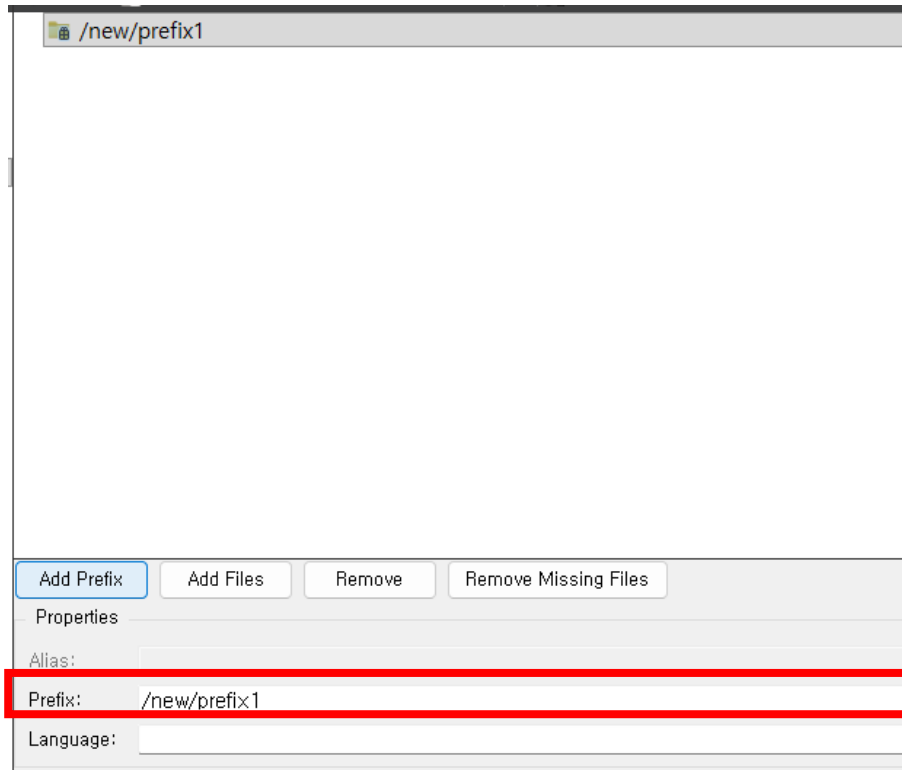
- 해당 .obj 파일을 현재 작업 중인 qt 디렉터리로 복사

Panel App 구현



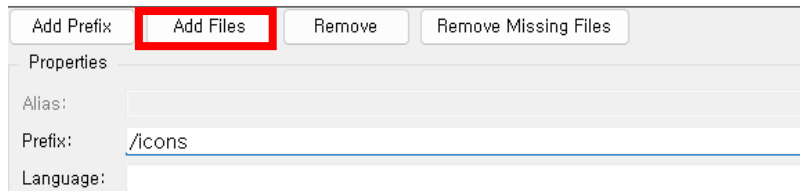
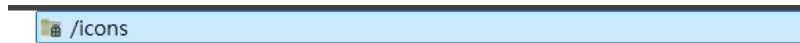
- icons.qrc 오른쪽 클릭
- Open in Editor클릭

Panel App 구현



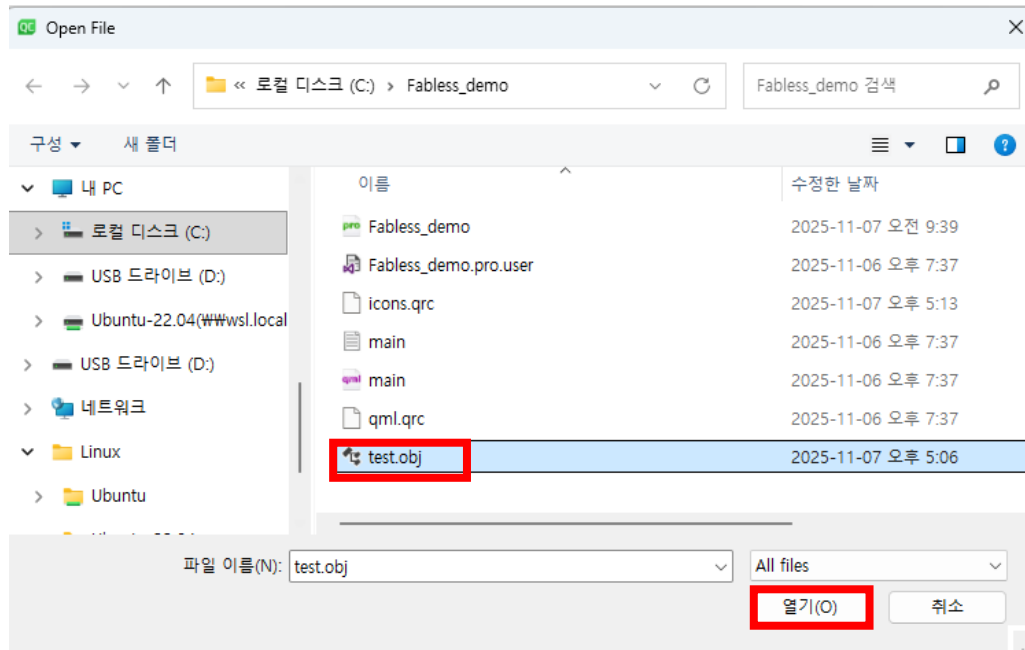
- Add Prefix 선택
- Prefix 를 /new/prefix1 -> /icons 로 수정

Panel App 구현



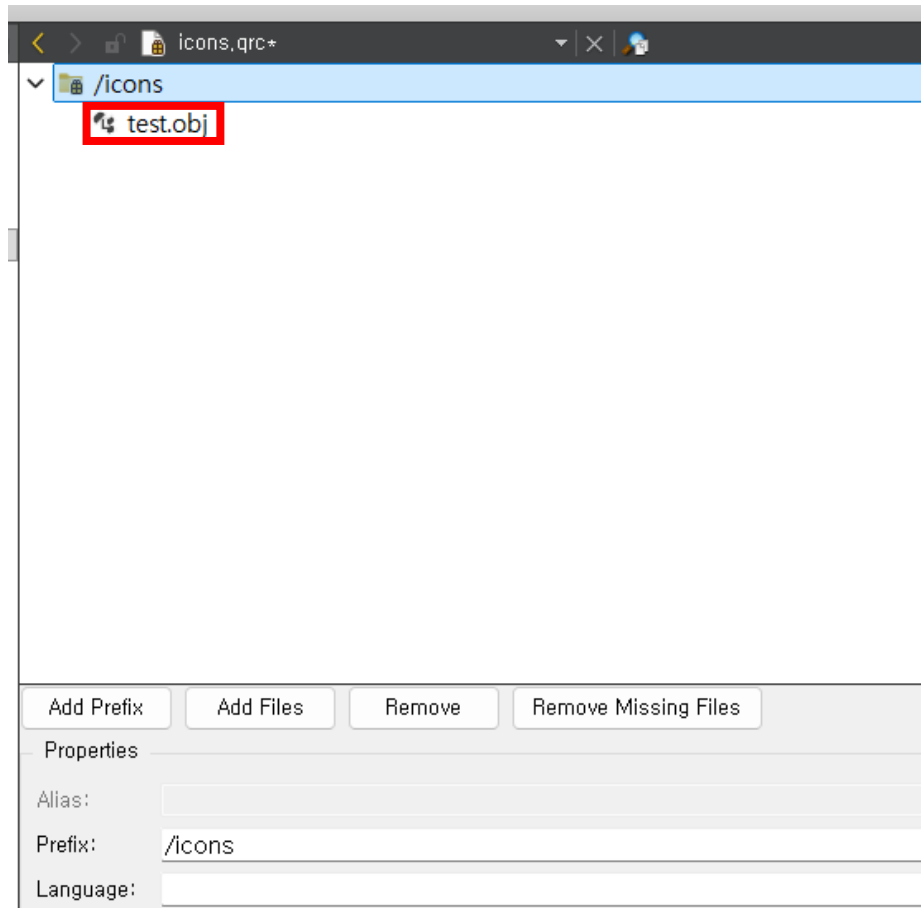
- Add Files 선택

Panel App 구현

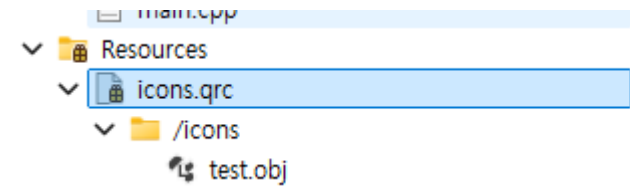


- test.obj 선택
- 열기 클릭

Panel App 구현



- 해당 화면에서 ctrl+s 입력
- 아래와 같이 생성되는지 확인



Panel App 구현

- Sources/main.cpp 코드 수정

```
#include <QGuiApplication>
#include <QQmlApplicationEngine>
#include <QTcpSocket>
#include <QTimer>
#include <QQmlContext>
#include <QDebug>
#include <QtGui/QFont>
#include <QtGui/QFontDatabase>
#include <QFile>
#include <QJsonDocument>
#include <QJsonObject>
#include <QCommandLineParser>
#include <QCommandLineOption>
#include <QSurfaceFormat>
#include <QGuiApplication>
#include <QQmlApplicationEngine>
#include <QQuickView>
```

```
class TcpClient : public QObject {
    Q_OBJECT
public:
    TcpClient() {
        connect(&socket, &QTcpSocket::connected, this, [](){
            qDebug() << "✅ 서버에 연결 성공!";
        });
        connect(&socket, &QTcpSocket::disconnected, this, [](){
            qDebug() << "⚠️ 서버와 연결이 끊어졌습니다.";
        });
        connect(&socket, &QTcpSocket::readyRead, this,
            &TcpClient::checkForData);
    }

    Q_INVOKABLE void connectTo(const QString& ip, int port) {
        if (socket.isOpen() || socket.state() ==
            QAbstractSocket::ConnectingState)
            socket.disconnectFromHost();
        qDebug() << "➡️ 서버 접속 시도:" << ip << port;
        socket.connectToHost(ip, port);
    }
}
```

Panel App 구현

- Sources/main.cpp 코드 수정

```
void connectFromConfig() {
    QFile file(":/icons/config.json");
    // qrc 기본값
    if (file.open(QIODevice::ReadOnly)) {
        auto json =
        QJsonDocument::fromJson(file.readAll()).object
        ();
        const QString ip =
        json.value("server_ip").toString();
        const int port =
        json.value("server_port").toInt();
        connectTo(ip, port);
    } else {
        qWarning() << "config.json(qrc) 열
기 실패";
    }
}

signals:
    void showCarsForLane(int lane); // lane
번호 전달
```

```
private slots:
    void checkForData() {
        if (socket.bytesAvailable() > 0) {
            QByteArray data = socket.readAll();

            if (data.contains("object_detected_lane_1")) emit
            showCarsForLane(1);
                else if (data.contains("object_detected_lane_2"))
            emit showCarsForLane(2);
                else if (data.contains("object_detected_lane_3"))
            emit showCarsForLane(3);
                else if (data.contains("object_detected_lane_4"))
            emit showCarsForLane(4);
                else if (data.contains("object_detected_lane_5"))
            emit showCarsForLane(5);
        }
    }

private:
    QTcpSocket socket;
    QTimer timer;
};
```

Panel App 구현

- Sources/main.cpp 코드 수정

```
int main(int argc, char *argv[])
{
    QGuiApplication app(argc, argv);

    //qt 검증을 위해 eglfs 옵션은 잠시 주석 처리
    //QSurfaceFormat format;
    //format.setRenderableType(QSurfaceFormat::OpenGL);
    //format.setVersion(2, 0);
    //QSurfaceFormat::setDefaultFormat(format);

    // --- 커맨드라인 파서 설정 ---
    QCommandLineParser parser;
    parser.setApplicationDescription("Lane telemetry
client");
    parser.addHelpOption();
    QCommandLineOption ipOpt({"i","ip"}, "Server IP
address", "ip");
    QCommandLineOption portOpt({"p","port"}, "Server port",
"port");
    parser.addOption(ipOpt);
    parser.addOption(portOpt);
    parser.process(app);

    QQmlApplicationEngine engine;

    QFontDatabase::addApplicationFont(":/fonts/DejaVuSans.ttf");
    app.setFont(QFont("DejaVu Sans"));
```

```
TcpClient client;
engine.rootContext()-
>setContextProperty("tcpClient", &client);

// 인자 우선, 없으면 qrc 설정 사용
const QString ipArg = parser.value(ipOpt);
bool ok = false;
int portArg = parser.value(portOpt).toInt(&ok);
if (!ipArg.isEmpty() && ok && portArg > 0) {
    client.connectTo(ipArg, portArg);
} else {
    client.connectFromConfig();
}

engine.load(QUrl(QStringLiteral("qrc:/main.qml")));
return app.exec();
}

#include "main.moc"
```

Panel App 구현

- qml.qrc/main.qml 코드 수정

```
import QtQuick 2.12
import QtQuick.Controls 2.12
import QtQuick.Scene3D 2.12
import Qt3D.Core 2.12
import Qt3D.Render 2.12
import Qt3D.Input 2.12
import Qt3D.Extras 2.12

ApplicationWindow {
    id: win
    width: 800
    height: 400
    visible: true
    title: "Lane 3D Demo"

    property real yaw: 0
    property real pitch: 10
    property real zoom: 30
    property var laneX: [-7.5, -2.5, 2.5, 7.5]
```

```
function updateCameraPosition() {
    let radYaw = yaw * Math.PI / 180
    let radPitch = pitch * Math.PI / 180
    let x = zoom * Math.sin(radYaw) *
Math.cos(radPitch)
    let y = zoom * Math.sin(radPitch)
    let z = zoom * Math.cos(radYaw) *
Math.cos(radPitch)
    camera.position = Qt.vector3d(x, y, z)
}
property var lastSeenMs: ({1:0, 2:0, 3:0, 4:0, 5:0})
// C++ setContextProperty("tcpClient", &client)
Connections {
    target: tcpClient
    function onShowCarsForLane(lane) {
        var now = Date.now()
        lastSeenMs[lane] = now
        if (lane === 1) carEnt1.enabled = true
        if (lane === 2) carEnt2.enabled = true
        if (lane === 3) carEnt3.enabled = true
        if (lane === 4) carEnt4.enabled = true
        if (lane === 5) carEnt5.enabled = true
    }
}
```


Panel App 구현

- qml.qrc/main.qml 코드 수정

```
Item {
    anchors.fill: parent

    Scene3D {
        anchors.fill: parent
        aspects: ["input", "logic", "render"]

        Entity {
            id: rootEntity

            Camera {
                id: camera
                position: Qt.vector3d(0, 20, 40)
                viewCenter: Qt.vector3d(0, 0, 0)
                upVector: Qt.vector3d(0, 1, 0)
            }
        }
    }
}
```

```
        components: [
            RenderSettings {
                activeFrameGraph:
ForwardRenderer {
                camera: camera
                clearColor: "#1a1a1a"
            },
            InputSettings {}
        ]

        // 조명
        Entity {
            components: [
                PointLight { color: "white";
intensity: 3; constantAttenuation: 1.0 },
                Transform { translation:
Qt.vector3d(0, 10, 10) }
            ]
        }
    ]
}
```

Panel App 구현

- qml.qrc/main.qml 코드 수정

```
// 도로
Entity {
    components: [
        PlaneMesh { width: 20; height: 80 },
        PhongMaterial { diffuse: "#303030"; shininess: 10; specular: "#555555" },
        Transform { rotation: fromAxisAndAngle(Qt.vector3d(1, 0, 0), 0); translation:
Qt.vector3d(0, -1.0, 0) }
    ]
}

// 차선 4개
Entity { components: [ CuboidMesh { xExtent: 0.3; yExtent: 0.2; zExtent: 60 },
PhongMaterial { diffuse: "white" }, Transform { translation: Qt.vector3d(-7.5, -1.0, -10) } ] }
Entity { components: [ CuboidMesh { xExtent: 0.3; yExtent: 0.2; zExtent: 60 },
PhongMaterial { diffuse: "white" }, Transform { translation: Qt.vector3d(-2.5, -1.0, -10) } ] }
Entity { components: [ CuboidMesh { xExtent: 0.3; yExtent: 0.2; zExtent: 60 },
PhongMaterial { diffuse: "white" }, Transform { translation: Qt.vector3d( 2.5, -1.0, -10) } ] }
Entity { components: [ CuboidMesh { xExtent: 0.3; yExtent: 0.2; zExtent: 60 },
PhongMaterial { diffuse: "white" }, Transform { translation: Qt.vector3d( 7.5, -1.0, -10) } ] }
```

Panel App 구현

- qml.qrc/main.qml 코드 수정

```
// 자차
Entity {
    components: [
        SceneLoader {
            id: carModel_my
            source: "qrc:/icons/test.obj"
            onStatusChanged: {
                if (status === SceneLoader.Ready) console.log("자차
모델 로드 완료")
                else if (status === SceneLoader.Error)
console.error("자차 모델 로드 실패")
            }
        },
        Transform {
            translation: Qt.vector3d(0, -0.9, 13)
            scale3D: Qt.vector3d(0.03, 0.03, 0.03)
            rotation: fromAxisAndAngle(Qt.vector3d(0, 1, 0), 90)
        }
    ]
}
```

Panel App 구현

- qml.qrc/main.qml 코드 수정

```
// 주변 차량들 (처음에 로드해두고 꺼둠: enabled=false)
Entity {
    id: carEnt1
    enabled: true
    components: [
        SceneLoader { id: carModel1; source: "qrc:/icons/test.obj" },
        Transform { translation: Qt.vector3d(-4.8, -1, -20); scale3D:
Qt.vector3d(0.03,0.03,0.03); rotation: fromAxisAndAngle(Qt.vector3d(0,1,0), -90) }
    ]
}
Entity {
    id: carEnt2
    enabled: true
    components: [
        SceneLoader { id: carModel2; source: "qrc:/icons/test.obj" },
        Transform { translation: Qt.vector3d(-4.8, -1, 2); scale3D:
Qt.vector3d(0.03,0.03,0.03); rotation: fromAxisAndAngle(Qt.vector3d(0,1,0), -90) }
    ]
}
```

Panel App 구현

- qml.qrc/main.qml 코드 수정

```
Entity {
    id: carEnt3
    enabled: true
    components: [
        SceneLoader { id: carModel3; source: "qrc:/icons/test.obj" },
        Transform { id: carTransform_1; translation: Qt.vector3d( 4.8, -1, 2);
scale3D: Qt.vector3d(0.03,0.03,0.03); rotation: fromAxisAndAngle(Qt.vector3d(0,1,0), 90) }
    ]
}
Entity {
    id: carEnt4
    enabled: true
    components: [
        SceneLoader { id: carModel4; source: "qrc:/icons/test.obj" },
        Transform { id: carTransform_2; translation: Qt.vector3d( 4.8, -1, -20);
scale3D: Qt.vector3d(0.03,0.03,0.03); rotation: fromAxisAndAngle(Qt.vector3d(0,1,0), 90) }
    ]
}
```

Panel App 구현

- qml.qrc/main.qml 코드 수정

```
Entity {
    id: carEnt5
    enabled: true
    components: [
        SceneLoader { id: carModel5; source: "qrc:/icons/test.obj" },
        Transform { translation: Qt.vector3d(0, -0.9, -5); scale3D:
Qt.vector3d(0.03,0.03,0.03); rotation: fromAxisAndAngle(Qt.vector3d(0,1,0), 90) }
    ]
}

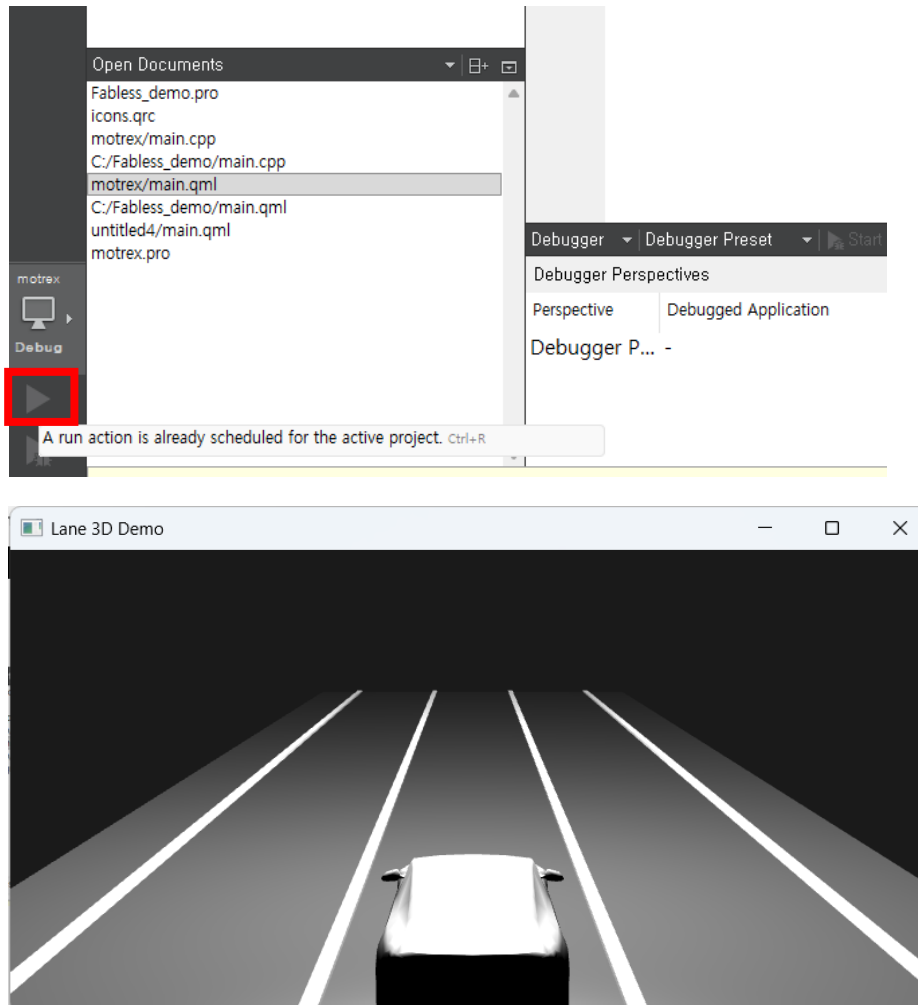
// 1초 뒤 끄기 (enabled=false). 더 이상 source="" 하지 않음!
Timer {
    interval: 200
    repeat: true
    running: true
    onTriggered: {
        var now = Date.now()
        if (now - lastSeenMs[1] > 1000) carEnt1.enabled = false
        if (now - lastSeenMs[2] > 1000) carEnt2.enabled = false
        if (now - lastSeenMs[3] > 1000) carEnt3.enabled = false
        if (now - lastSeenMs[4] > 1000) carEnt4.enabled = false
        if (now - lastSeenMs[5] > 1000) carEnt5.enabled = false
    }
}
```

Panel App 구현

- qml.qrc/main.qml 코드 수정

```
    }  
    }  
    }  
    }  
    }  
    Component.onCompleted: updateCameraPosition()  
}
```

Panel App 구현



- 작성이 완료되면 run 실행을 통해 검증
- 다음과 같은 qt 화면 확인

Panel App 구현 확인

```
int main(int argc, char *argv[])
{
    QGuiApplication app(argc, argv);

    //QSurfaceFormat format;
    //format.setRenderableType(QSurfaceFormat::OpenGL);
    //format.setVersion(2, 0);
    //QSurfaceFormat::setDefaultFormat(format);

    // --- 커맨드라인 파서 설정 ---
    QCommandLineParser parser;
    parser.setApplicationDescription("Lane telemetry client");
    parser.addHelpOption();
    QCommandLineOption ipOpt({"i","ip"}, "Server IP address", "ip");
    QCommandLineOption portOpt({"p","port"}, "Server port", "port");
```

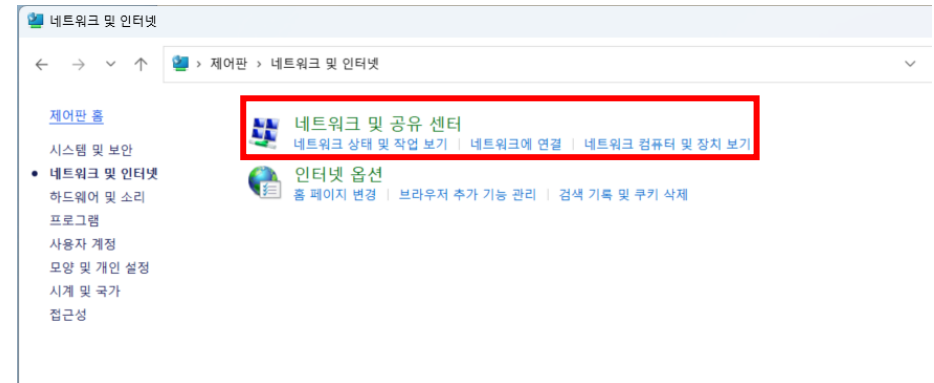
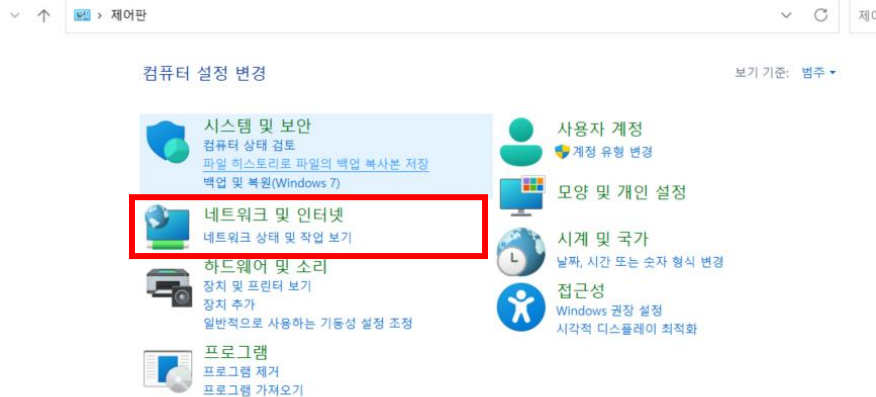
```
int main(int argc, char *argv[])
{
    QGuiApplication app(argc, argv);

    QSurfaceFormat format;
    format.setRenderableType(QSurfaceFormat::OpenGL);
    format.setVersion(2, 0);
    QSurfaceFormat::setDefaultFormat(format);

    // --- 커맨드라인 파서 설정 ---
    QCommandLineParser parser;
    parser.setApplicationDescription("Lane telemetry client");
    parser.addHelpOption();
    QCommandLineOption ipOpt({"i","ip"}, "Server IP address", "ip");
    QCommandLineOption portOpt({"p","port"}, "Server port", "port");
    parser.addOption(ipOpt);
```

- main.cpp에 주석 해제

Panel App 구현 확인

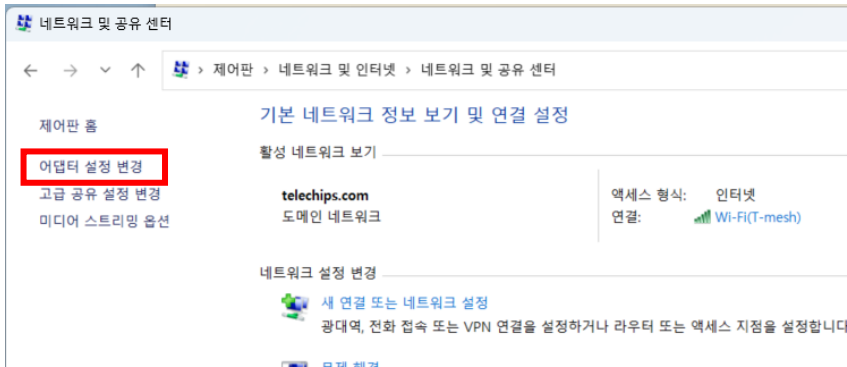


• Ethernet Set Up

▪ Host PC Network Configuration

➤ 제어판 -> 네트워크 및 인터넷 -> 네트워크 및 공유 센터 -> 어댑터 설정 변경

Panel App 구현 확인



• Ethernet Set Up

▪ Host PC Network Configuration

➤ 제어판 -> 네트워크 및 인터넷 -> 네트워크 및 공유 센터 -> 어댑터 설정 변경

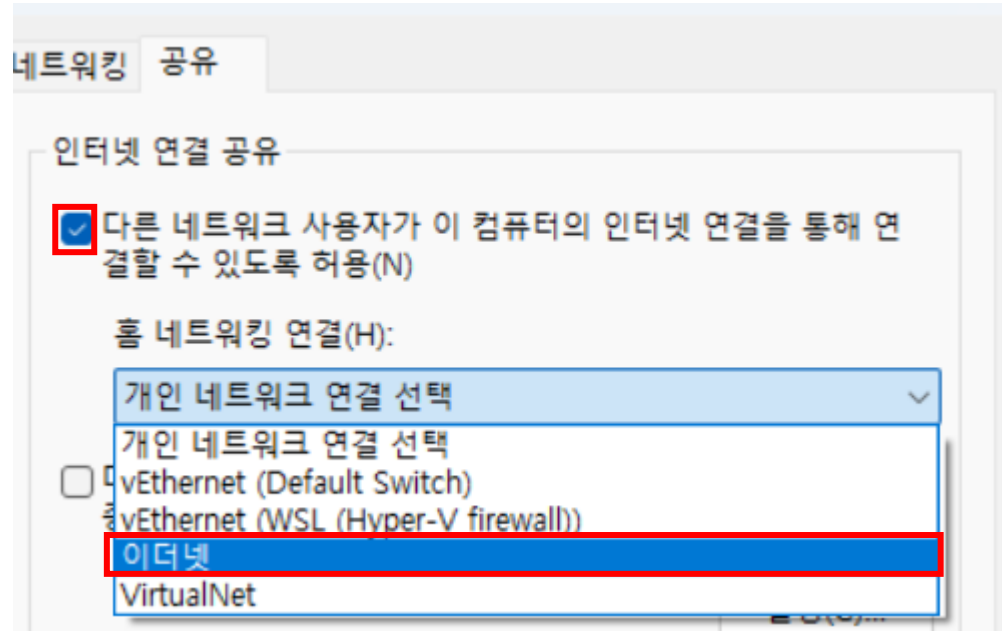
Panel App 구현 확인



- Ethernet Set Up

- Host PC Network Configuration
 - 이더넷 선택 -> 속성 -> 공유

Panel App 구현 확인

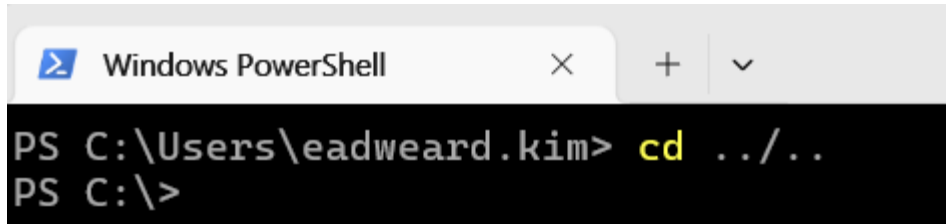


- Ethernet Set Up

- 인터넷 연결 공유 옵션 활성화
- 홈 네트워킹 연결 탭에서 "이더넷" 선택 -> 확인 선택

Panel App 구현 확인

- QT panel 보드로 전송
 - D3-G보드에서 ifconfig 명령어로 ip 확인
 - Power shell 실행
 - cd .. 명령어를 통해 c 드라이브로 이동

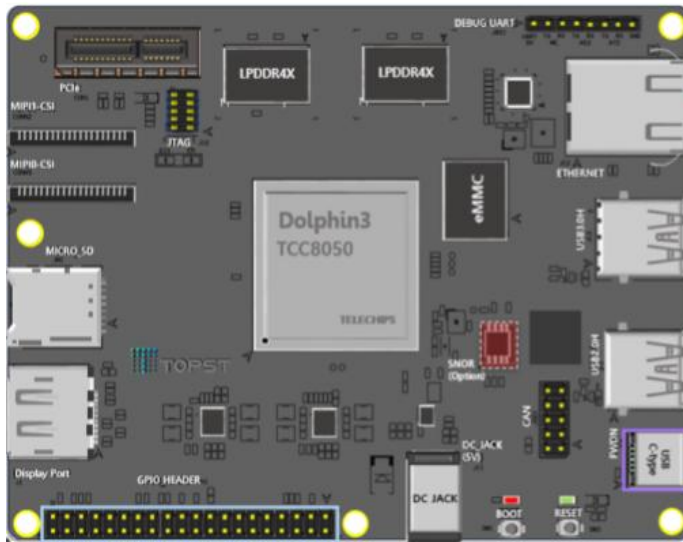


```
Windows PowerShell
PS C:\Users\eadweard.kim> cd ../../
PS C:\>
```

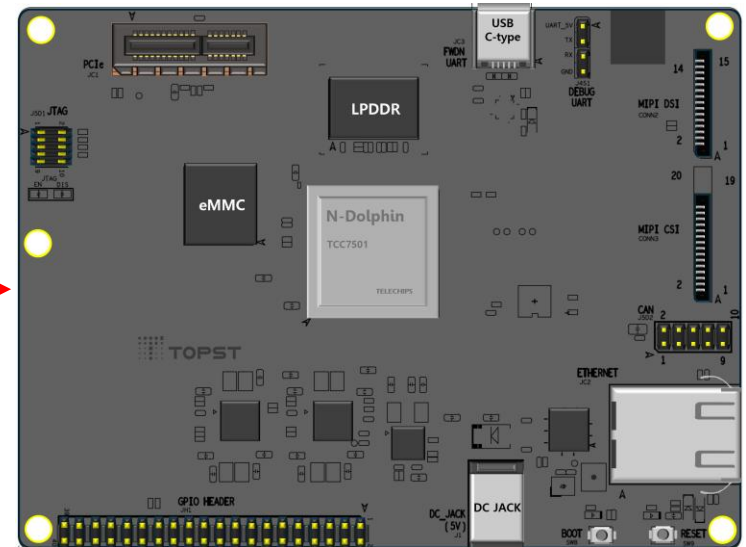
- \$ scp -r Fabless_demo topst@{확인한 ip}:/home/topst
- 암호 topst입력

Panel App 구현 확인

- D3-G 보드와 AI-G를 랜선으로 연결

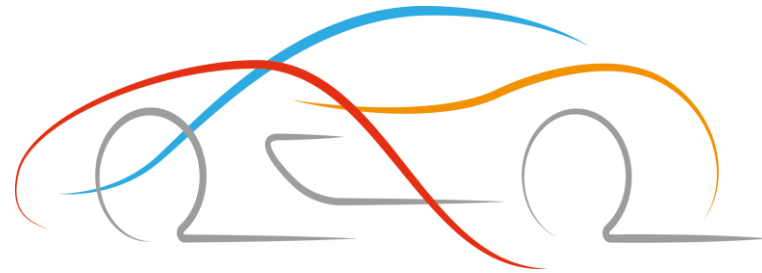


Ethernet
Cable



Panel App 구현 확인

- D3-G 보드의 IP 설정
 - `$ sudo ifconfig eth0 192.168.0.101 up`
- Fabless 앱 디렉터리로 이동
 - `$ cd Fabless_Education_blink/Step3/fabless`
 - `$ sudo chmod 755 *`
 - `$ sudo make clean`
 - `$ sudo qmake`
 - `$ sudo make`
 - `$ sudo ./fabless -i 192.168.0.100 -p 9999`
 - AI-G 추론 시작 및 D3-G 패널 앱 정상 동작 확인



Thank You